# JavaCPUtils: Architecture, Implementation & Publishing Guide

**Author:** shaheed shaik

**GitHub:** https://github.com/shaheed3515/javacputils

**Release:** `v1.0.0` (JitPack: `com.github.shaheed3515:javacputils:v1.0.0`)

---

## Table of Contents

- Executive Summary & Motivation
- Language Mechanics: Java vs. C++ in Competitive Programming
- Architecture of Core Library Classes

- Generic Pair (`Pair<F, S>`)

- High-Performance Primitive `IntPair`

- High-Performance Primitive `LongPair`

- Sorting, Comparison & Java Memory Deep Dive

- Why Primitive Specialization is 3x Faster

- The Subtraction Overflow Trap

- PriorityQueue & Dijkstra Min-Heap Integration

- The Online Judge Dilemma & Automated Bundler

- Why Online Judges Reject External Libraries

- How `bundle Solution.java` Solves It

- Multi-Channel Distribution Guide

- Channel 1: 1-Click Zero-Config Installation (Beginners/Students)

- Channel 2: Gradle & Maven via JitPack (Developers)

- Channel 3: Direct Standalone JAR Download

- Publishing Pipeline: From Code to JitPack CDN
- Roadmap for `JavaCPUtils`

---

## 1. Executive Summary & Motivation

In competitive programming (CP) and algorithm contests (Codeforces, LeetCode, AtCoder, CSES), speed of implementation and execution speed are paramount.

C++ developers take `std::pair` for granted:

```
pair<int, int> p = {10, 20};
p.first += 5;
cout << p.first << " " << p.second;
```

Java developers, however, have historically faced multiple barriers:

- Java has no standard `Pair` in `java.util`.
- `Map.Entry` is verbose and immutable.
- Java records (`record Pair<F, S>(F first, S second)`) use accessor methods `p.first()` instead of public fields and cannot be mutated in place.
- Generic `Pair<Integer, Integer>` introduces severe autoboxing and pointer-chasing overhead, which can cause Time Limit Exceeded (TLE) on tight constraints ($N = 2 \times 10^5$, $1.0\text{s}$).

**`JavaCPUtils`** was created to eliminate this gap completely by providing:

- Direct public fields (`p.first`, `p.second`) for the exact C++ developer experience.
- Natural lexicographical comparison out-of-the-box (`Comparable<Pair>`).
- Primitive-specialized `IntPair` and `LongPair` classes delivering up to **3x faster execution** and **60% memory reduction**.
- An automated **Bundler CLI** that allows writing clean `import cp.*;` locally while producing single-file code for online judges.
- Global 1-click installers so students and teammates never have to configure build tools.

---

## 2. Language Mechanics: Java vs. C++ in Competitive Programming

| Feature | C++ (`std::pair`) | Standard Java | JavaCPUtils |
|---|---|---|---|
| **Field Access** | `p.first`, `p.second` | `p.getKey()`, `p.first()` | `p.first`, `p.second` |
| **Field Mutation** | Direct (`p.first = 5;`) | Immutable or setter method | Direct (`p.first = 5;`) |
| **Object Creation** | `make_pair(a, b)` | `new AbstractMap.SimpleEntry<>(a, b)` | `Pair.of(a, b)` or `new Pair<>(a, b)` |
| **Type Inference** | `auto p = ...` | `var p = ...` (Java 10+) | `var p = Pair.of(...)` |
| **Lexicographical Sort** | Built-in (`operator<`) | Manual `Comparator` lambda required | Automatic (`implements Comparable`) |
| **Primitive Speed** | Native primitive `pair<int, int>` | Boxed `Pair<Integer, Integer>` (slow) | Native `IntPair` & `LongPair` (ultra-fast) |
| **Package Import** | `#include <utility>` | None | `import cp.*;` |

---

## 3. Architecture of Core Library Classes

### Generic Pair (`Pair<F, S>`)

Located in package `cp`:

```
public class Pair<F, S> implements Comparable<Pair<F, S>>, Serializable, Cloneable {
    public F first;
    public S second;

    public Pair() { ... }
    public Pair(F first, S second) { ... }
    public Pair(Pair<? extends F, ? extends S> other) { ... }
```

```
    public static <F, S> Pair<F, S> of(F first, S second) { return new Pair<>(first, second);
}
    public static <F, S> Pair<F, S> make(F first, S second) { return new Pair<>(first,
second); }

    public void set(F first, S second) { this.first = first; this.second = second; }
    public Pair<S, F> swap() { return new Pair<>(this.second, this.first); }

    @Override
    public int compareTo(Pair<F, S> other) { ... } // Lexicographical comparison

    public static <F extends Comparable<? super F>, S> Comparator<Pair<F, S>>
comparingFirst() { ... }
    public static <F, S extends Comparable<? super S>> Comparator<Pair<F, S>>
comparingSecond() { ... }
}
```

### High-Performance Primitive `IntPair`

- Fields: `public int first; public int second;`
- Memory per instance: **24 bytes** (vs. ~64 bytes for `Pair<Integer, Integer>`).
- Hashing: `hashCode()` uses direct integer arithmetic (`31 * first + second`) with zero array allocation.
- Comparison: Uses `Integer.compare(first, other.first)` to remain completely immune to integer overflow.

### High-Performance Primitive `LongPair`

- Fields: `public long first; public long second;`
- Essential for graph algorithms (e.g., Dijkstra's shortest path) where cumulative path weights exceed $2 \times 10^9$ and require 64-bit integer precision.

---

## 4. Sorting, Comparison & Java Memory Deep Dive

### Why Primitive Specialization is 3x Faster

In standard Java generics, primitive types cannot be type arguments directly, requiring boxed objects (`Integer`, `Long`):

```
Generic Pair<Integer, Integer> in RAM:
[ Pair Object (~24 bytes) ]
   ├── Pointer to first  ──> [ Integer Object (~16 bytes) ]
```

```
   └── Pointer to second ──> [ Integer Object (~16 bytes) ]
Total: ~56 - 64 bytes per pair.

Primitive IntPair in RAM:
[ IntPair Object (~24 bytes) ]
   ├── int first  (4 bytes contiguous)
   └── int second (4 bytes contiguous)
Total: 24 bytes per pair (Nearly 3x memory reduction).
```

### Performance Consequences:

- **CPU Cache Locality:** `IntPair` stores `first` and `second` adjacent in memory. Loading `first` into CPU cache lines automatically loads `second`.
- **Garbage Collection (GC) Pressure:** Sorting $2 \times 10^5$ items creates **600,000 objects** with boxed `Pair`, but only **200,000 objects** with `IntPair`.
- **Autoboxing Overhead:** Avoids millions of hidden calls to `Integer.valueOf()` and `intValue()`.

### The Subtraction Overflow Trap

Many competitive programmers mistakenly sort using subtraction:

```
// ❌ DANGEROUS BUG IN JAVA:
list.sort((a, b) -> a.second - b.second);
```

If `a.second = -2,000,000,000` and `b.second = 2,000,000,000`, the subtraction underflows to `+294,967,296`. Java's TimSort will misorder the elements or throw `IllegalArgumentException: Comparison method violates its general contract!`.

**JavaCPUtils guarantees safety** by using `Integer.compare()` and `Long.compare()` throughout all `compareTo` and built-in `Comparator` implementations:

```
// ✅ SAFE & OVERFLOW-PROOF:
@Override
public int compareTo(IntPair other) {
    if (this.first != other.first) {
        return Integer.compare(this.first, other.first);
    }
    return Integer.compare(this.second, other.second);
}
```

### PriorityQueue & Dijkstra Min-Heap Integration

Because `Pair`, `IntPair`, and `LongPair` implement `Comparable`, they integrate natively with Java

collections:

```
// Dijkstra's Shortest Path Algorithm:
PriorityQueue<LongPair> pq = new PriorityQueue<>();
pq.add(new LongPair(0L, startNode)); // (distance, node)

while (!pq.isEmpty()) {
    LongPair top = pq.poll();
    long d = top.first;
    int u = (int) top.second;
    ...
}
```

---

## 5. The Online Judge Dilemma & Automated Bundler

### Why Online Judges Reject External Libraries

When submitting code to Codeforces, LeetCode, CSES, or AtCoder:

- The execution sandbox is strictly offline and isolated.
- The judging servers only contain standard JDK classes (`java.util.*`, `java.io.*`).
- Submitting `import cp.*;` directly produces: `error: package cp does not exist`.

### How `bundle Solution.java` Solves It

`JavaCPUtils` includes a dedicated bundling utility (`scripts/bundle.py` and `scripts/bundle`).

#### Workflow:

- Write clean solution code locally:

```java

import cp.*;

import java.util.*;

public class Solution {

public static void main(String[] args) {

IntPair p = new IntPair(10, 20);
```

System.out.println(p.first + p.second);

}

}

```

- Run the bundle command:

```bash

bundle Solution.java

```

- The tool:

- Removes `import cp.*;`.

- Detects all used library classes (`IntPair`, `Pair`, `LongPair`).

- Inlines them as package-private classes at the bottom of `Solution_bundled.java`.

- Adjusts class accessibility so `javac Solution_bundled.java` compiles under any file name.

- Copy-paste `Solution_bundled.java` into Codeforces or LeetCode for an immediate **Accepted (AC)** verdict!

---

## 6. Multi-Channel Distribution Guide

### Channel 1: 1-Click Zero-Config Installation (Beginners/Students)

Designed for students and peers who do not use Maven or Gradle.

### Linux & macOS:

```
curl -fsSL https://raw.githubusercontent.com/shaheed3515/javacputils/main/scripts/install.sh | bash
```

*What it does:* Downloads `javacputils.jar`, sets up `CLASSPATH` in `~/.bashrc` / `~/.zshrc`, and installs the `bundle` command into `PATH`.
```

### Windows (PowerShell):

```
irm https://raw.githubusercontent.com/shaheed3515/javacputils/main/scripts/install.bat
-OutFile install.bat; .\install.bat
```

*What it does:* Downloads `javacputils.jar` to `%USERPROFILE%\.javacputils`, updates the user `CLASSPATH` and `Path` environment variables in Windows Registry.

---

### Channel 2: Gradle & Maven via JitPack (Developers)

### Gradle (`build.gradle`):

```
repositories {
    mavenCentral()
    maven { url 'https://jitpack.io' }
}

dependencies {
    implementation 'com.github.shaheed3515:javacputils:v1.0.0'
}
```

### Maven (`pom.xml`):

```
<repositories>
    <repository>
        <id>jitpack.io</id>
        <url>https://jitpack.io</url>
    </repository>
</repositories>

<dependencies>
    <dependency>
        <groupId>com.github.shaheed3515</groupId>
        <artifactId>javacputils</artifactId>
        <version>v1.0.0</version>
    </dependency>
</dependencies>
```

---

### Channel 3: Direct Standalone JAR Download

Pre-compiled binary assets are attached to every GitHub Release:

- `javacputils-1.0.0.jar` (Binary library)
- `javacputils-1.0.0-sources.jar` (Source code for IDEs)
- `javacputils-1.0.0-javadoc.jar` (API Documentation)

Download from: github.com/shaheed3515/javacputils/releases/tag/v1.0.0

---

## 7. Publishing Pipeline: From Code to JitPack CDN

The release process follows modern automated artifact publishing:

```
[ Local Java Code (src/main/java/cp) ]
                  |
                  ▼  (mvn clean test package)
[ Verified Artifacts (JAR, Sources, Javadoc) ]
                  |
                  ▼  (git tag v1.0.0 && git push)
[ GitHub Repository & Release (shaheed3515/javacputils) ]
                  |
                  ▼  (Triggered via HTTP request)
[ JitPack Build Engine (JitPack CDN) ]
                  |
                  ▼  (Global distribution)
[ Any Developer / Laptop in the World ]
```

---

## 8. Roadmap for `JavaCPUtils`

**v1.0.0**: `Pair<F, S>`, `IntPair`, `LongPair`, Bundler tool, 1-Click installers, JitPack release.

**v1.1.0**: `Triple<A, B, C>` and primitive `IntTriple` / `LongTriple`.

**v1.2.0**: `FastScanner` and `FastWriter` for high-throughput I/O (up to 5x faster than `Scanner` and `System.out.println`).

**v1.3.0**: Core algorithmic data structures:

- Disjoint Set Union (DSU with union by rank and path compression)

- Fenwick Tree (Binary Indexed Tree / BIT)

- Segment Tree with point & range updates